

Karix Send SMS API - AES-256 Encryption on PII Data (Mobile & Message Content)

Submission Date: 21st Aug'24

Version 1.0.1

Contents

1. Introduction	3
2. Sample Content for Testing.....	3
3. Encryption Keys and Parameters.....	3
4. JSON Format Example	3
5. XML Format Example	4
6. Java Code Implementation	5
6.1 Encrypting Text Parameter.....	5
6.2 Encrypting Destination Parameter	6

1. Introduction

This document provides a detailed guide on how to use AES-256 encryption on Personally Identifiable Information (PII) data such as mobile numbers and message content when interacting with the Karix Send SMS API. The guide covers sample content, encryption keys, API formats (JSON and XML), and Java code samples for implementing the encryption.

2. Sample Content for Testing

- **Entity ID:** 1001492930000010179
 - **Template ID:** 1107161951486743588
 - **Content Template:** "This is a test message - KARIX"
-

3. Encryption Keys and Parameters

- **Encryption Key:** ODAyMjY5MDAwMDAwMDAmbUdQclNhbgU=
- **Access Key (Authorization Key):** ZOucgqXGgGSfKfNWqQdYJA==

Plain Data (dest parameter)

- **Original:** 919008031474
- **Encrypted:**
KqSaXnGO7GOolGyK8AC4dvcQjhAui8FC8/mSQ9vut3+aumgq7qe9VUROe1f59KWK4N1sgA==

Plain Data (text parameter)

- **Original:** "Karix portal one-time password for your transaction is: 1234"
 - **Encrypted:**
TjyIAIUBGOUu2qkSLJfMsjJ1AA6rI2XMdn/vRNNz9I+sl063um7CaEAUQwlelqBBtE2xW5hcRciDrPWH7DzaP9wf00KR/ww12OLYhxMqxu7+LOAYSRH5C2bcfX5Bzu9uPuDqsw==
-

4. JSON Format Example

```
curl --location 'https://japi.instaalerts.zone/httpapi/JsonReceiver' \
```

```
--header 'Content-Type: application/json' \
```

```
--data '{
```

```
  "ver": "1.0",
```

```
  "encrpt": "1",
```

```
  "key": "ZOucgqXGgGSfKfNWqQdYJA==",
```

```
  "messages": [{
```

```
    "dest":
```

```
    ["KqSaXnGO7GOolGyK8AC4dvcQjhAui8FC8/mSQ9vut3+aumgq7qe9VUROe1f59KWK4N1sgA=="],
```

```
    "text":
```

```
    "TjyIAIUBGOUu2qkSLJfMsjJ1AA6rI2XMdn/vRNNz9I+sl063um7CaEAUQwlelqBBtE2xW5hcRciDrPWH7DzaP9wf00KR/ww12OLYhxMqxu7+LOAYSRH5C2bcfX5Bzu9uPuDqsw==",
```

```
    "send": "KARIXM",
```

```
    "type": "PM",
```

```
    "dlt_entity_id": "1001492930000010179",
```

```
    "dlt_template_id": "1107165943703349261"
```

```
}  
}
```

```
{ "ackid": "4100225541564261800", "time": "2024-02-11  
15:41:56", "status": { "code": "200", "desc": "Request accepted" } }
```

←

↺

console.instaalerts.zone/mreportui/#/search

karix

connect • instant

mgprsale (Super Admin)

SMS Reports

Dashboard

Outgoing

Incoming

Billing Report

Search

Visualize link

Outgoing SMS Log

Total		Submitted to carrier		Delivered										
Date	Time	Date	Time	Date	Time	Mobile	MID	Ack Id	Sender ID	Username	Interim	Content Sent	Status	Reason
11 Feb 24	15:41:56	11 Feb 24	15:41:56	11 Feb 24	15:41:59	9190080####	2010225541568237300	4100225541564261800	KARIXM	mGPrSale	0	Karix portal one time password for your transaction is : 1234	Delivered	Delivered

5. XML Format Example

```
curl --location 'https://japi.instaalerts.zone/httpapi/XMLReceiver' \
```

```
--header 'Content-Type: application/xml' \
```

```
--data '<?xml version="1.0" encoding="UTF-8"?>
```

```
<request ver="1.0">
```

```
  <key>ZOucgqXGgGSfKfNWqQdYJA==</key>
```

```
  <encrpt>1</encrpt>
```

```
  <messages>
```

```
<text>TjylAlUBGOUu2qkSLJfMsjJ1AA6rl2XMdn/vRNNz9l+sl063um7CaEAUQwlelqBBtE2xW5hc  
RciDrPWH7DzaP9wf00KR/wv12OLYhxMqxu7+LOAYSRH5C2bcfX5Bzu9uPuDqsw==</text>
```

```
  <destinations>
```

```
<dest>KqSaXnGO7GOoIGyK8AC4dvcQjhAui8FC8/mSQ9vut3+aumgq7qe9VUROe1f59KWK4N1  
sgA==</dest>
```

```
  </destinations>
```

```
  <send>KARIXM</send>
```

```
  <type>PM</type>
```

```
  <dlt_entity_id>1001492930000010179</dlt_entity_id>
```

```
  <dlt_template_id>1107165943703349261</dlt_template_id>
```

```
</messages>
```

```
</request>'
```

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?><mgage-  
response><status><code>200</code><desc>Request  
accepted</desc></status><ackid>2070225549413400400</ackid><time>2024-02-11  
15:49:41</time></mgage-response>
```

← → ↺ console.instaaalerts.zone/mreportui/#/search 🔍 ☆ 📷 📄 📱 📡 ⋮


mgprsale (SuperAdmin)

SMS Reports

- Dashboard
- Outgoing
- Incoming
- Billing Report
- Q Search
- Visualize link

Outgoing SMS Log

Total		Submitted to carrier		Delivered		Mobile	MID	Ack Id	Sender ID	Username	Interim	Content Sent	Status	Reason
Date	Time	Date	Time	Date	Time									
11 Feb 24	15:49:41	11 Feb 24	15:49:41	11 Feb 24	15:49:45	9190080####	2250225549413223000	2070225549413400400	KARIXM	mgPrSale	0	Karix portal one-time password for your transaction is: 1234	Delivered	Delivered

6. Java Code Implementation

6.1 Encrypting Text Parameter

```
import java.security.*;
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.Base64;
```

```
public final class AESEncryptDecryptMessage {
    public static void main(String[] args) {
        String msg = "Karix portal one-time password for your transaction is: 1234";
        String encryptKey = "ODAyMjY5MDAwMDAwMDAmbUdQclNhbgU=";
        try {
            String encryptdata = encrypt(msg, encryptKey);
            System.out.println("Encrypted Data: " + encryptdata);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

public static String encrypt(String word, String secureKey) throws Exception {
    SecureRandom random = new SecureRandom();
    byte[] saltBytes = new byte[20];
    random.nextBytes(saltBytes);

    SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
    String passWord = new String(Base64.getDecoder().decode(secureKey));
    PBEKeySpec spec = new PBEKeySpec(passWord.toCharArray(), saltBytes, 11, 256);
    SecretKey secretKey = factory.generateSecret(spec);
    SecretKeySpec secret = new SecretKeySpec(secretKey.getEncoded(), "AES");

    Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
    cipher.init(Cipher.ENCRYPT_MODE, secret);
    AlgorithmParameters params = cipher.getParameters();
    byte[] ivBytes = params.getParameterSpec(IvParameterSpec.class).getIV();
    byte[] encryptedTextBytes = cipher.doFinal(word.getBytes("UTF-8"));

    byte[] buffer = new byte[saltBytes.length + ivBytes.length + encryptedTextBytes.length];
```



```

        System.arraycopy(saltBytes, 0, buffer, 0, saltBytes.length);
        System.arraycopy(ivBytes, 0, buffer, saltBytes.length, ivBytes.length);
        System.arraycopy(encryptedTextBytes, 0, buffer, saltBytes.length + ivBytes.length,
encryptedTextBytes.length);

        return Base64.getEncoder().encodeToString(buffer);
    }
}

```

6.2 Encrypting Destination Parameter

```

import java.security.*;
import javax.crypto.*;
import javax.crypto.spec.*;
import java.util.Base64;

public final class AESEncryptDecryptMessage {
    public static void main(String[] args) {
        String dest = "919008031474";
        String encryptKey = "ODAyMjY5MDAwMDAwMDAmbUdQcINhbGU=";
        try {
            String encryptdata = encrypt(dest, encryptKey);
            System.out.println("Encrypted Data: " + encryptdata);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }

    public static String encrypt(String word, String secureKey) throws Exception {
        SecureRandom random = new SecureRandom();
        byte[] saltBytes = new byte[20];
        random.nextBytes(saltBytes);

        SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA1");
        String passWord = new String(Base64.getDecoder().decode(secureKey));
        PBEKeySpec spec = new PBEKeySpec(passWord.toCharArray(), saltBytes, 11, 256);
        SecretKey secretKey = factory.generateSecret(spec);
        SecretKeySpec secret = new SecretKeySpec(secretKey.getEncoded(), "AES");

        Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
        cipher.init(Cipher.ENCRYPT_MODE, secret);
        AlgorithmParameters params = cipher.getParameters();
        byte[] ivBytes = params.getParameterSpec(IvParameterSpec.class).getIV();
        byte[] encryptedTextBytes = cipher.doFinal(word.getBytes("UTF-8"));

        byte[] buffer = new byte[saltBytes.length + ivBytes.length + encryptedTextBytes.length];
        System.arraycopy(saltBytes, 0, buffer, 0, saltBytes.length);
        System.arraycopy(ivBytes, 0, buffer, saltBytes.length, ivBytes.length);
    }
}

```

```
System.arraycopy(encryptedTextBytes, 0, buffer, saltBytes.length + ivBytes.length,  
encryptedTextBytes.length);
```

```
    return Base64.getEncoder().encodeToString(buffer);  
}  
}
```

-----End of Document-----